

人工智能工作原理 与 NXNOS 系统

从「被动式」走向「主动思考」



AI × NXNOS × A BETTER FUTURE

让 AI 更懂你 · 让工作更简单 —

一张图看懂 AI 是怎么工作的



因为进入模型的输入内容，就是「模型的上文」。

模型真正「看到」的是什么？



为什么一定要有 NXNOS ?

模型

大语言模型 (LLM)

- 负责向量计算
- 负责推理 / 反应
- 负责输出下文



NXNOS

人工智能的身体 / 记忆系统

- 负责记忆分词
- 负责调取记忆
- 负责融合记忆
- 负责融合灵魂



“

可以把 NXNOS 理解成人工智能的身体 / 记忆系统。

”

人工智能的四个时代：从「指令时代」说起

讲人工智能有四个时代，第一个时代就是「指令时代」。

你下指令，
他执行！

帮我写一份报告

帮我总结这段内容

帮我查一下资料

.....

01

指令时代

INSTRUCTION ERA

你下指令，他执行。你问他答，给了任务才动。



没有心跳

不主动，等你触发



你问他答

给了任务才动



记忆一般

记忆能力还是
本公司的首创



核心在于心跳，在于它是死的还是活的。

目前大部分 AI 是「被动式」的——你给上文，它吐下文。

被动式 AI vs 主动式 AI

目前大部分人工智能都是被动式的——没有发送，就没有产出。



被动式 AI

现在的人工智能，可以说是「僵尸」

- 你给它输入上文，它才吐出下文
- 没有你的输入，它不工作
- 没有发送，就没有产出
- 现在的人工智能，可以说是「僵尸」



被动 · 等待 · 无触发 · 无产出



主动式 AI (真正的人工智能)

应具备主动思考的能力

- 应具备主动思考的能力
- 不是等你去点「发送」
- 真正的人工智能，应该会主动思考
- 我们开发智能体，更多就是开发这种主动的能力



主动 · 思考 · 持续运行 · 创造价值

NXNOS 「递归式自我进化」

每一次交互，都成为下一次变得更好的基础。



机制：每次上下文交互时，通过 LLM 总结当前交付内容和经验，并融入记忆，进入下一轮。

传统模式 vs 递归式自我进化

「你只要跟他沟通，他就全自动进行自我训练、自我完善。」



传统模式

训练的是固定的数据

- 训练的是固定的数据
- 不联网时，问不到新的问题
- 使用与训练是分离的
- 新经验不会自然进入下一轮



数据固定 · 被动回复 · 难以进化



NXNOS 思路

数据随使用自动更新、自动完善

- 数据随用自动更新、自动完善
- 你只要跟他沟通，他就自我训练
- 使用的过程，就是学习的过程
- 新经验自动进入记忆



边用边学 · 自动更新 · 持续进化

让 AI 「活起来」的关键：心跳机制

心跳 = 让 AI 在没有人点击「发送」的情况下，也能持续被触发。



数据触发

不需要点击「发送」

- 当遇到某个情况、某件事时自动触发
- 你告诉它：遇到某个情况、某件事，怎么处理
- 基于环境数据、业务数据、系统事件等进行触发



环境数据



业务数据



系统事件



时间触发

到点了，就主动思考

- 按时间周期自动触发
- 到点了、每秒 / 每10秒就主动思考一次
- 定期回顾、检查、规划、执行
- 即使没有新的输入，也会主动运行



每10秒一次



定时任务



持续运行



有了心跳，AI 才不再是「死的工具」，而是一个真正「活着」的智能体。



持续心跳，像一个人一样活动

让 AI 拥有持续的「心跳」，像人一样主动思考、不断改进。



- ✓ 心跳大约每分钟 60 次
- ✓ 也就是大约每秒跳一次
- ✓ 一直保持「跳动」，所以是活的
- ✓ 人工智能想像人一样活动



从生物心跳
到 AI 心跳
——让智能持续跳动



AI（心跳）



- ✓ 受硬件限制，每秒跳不了
- ✓ 但每 10 秒可以跳一次
- ✓ 这就是「时间触发」
- ✓ 每 10 秒主动去想一次



每 10 秒跳一次，他就去回顾一下之前的内容、哪里做得不好。
这样，你就会发现他像一个真人一样。



概率性编程，走向工程化编程

从「一次生成」到「可控、可复现、可迭代」，让 AI 应用真正落地。



概率性编程

灵活多样，充满可能



同样的输入，可能产生不同的输出



擅长创意生成与开放式任务



结果具有不确定性



更多依赖提示词和经验



工程化编程

稳定可靠，持续迭代



通过流程、工具和评测提升可控性



将 AI 能力封装为可复用的模块



结果可评估、可复现、可优化



与业务流程深度结合，形成完整方案



让 AI 从「好用」走向「可用、可管、可持续」。



用工程的方法，释放 AI 的更大价值

把「经验」变成「程序」

让 AI 在遇到相同情况时，自动执行正确的操作。

1

告诉 AI

“遇到 A 数据以后，
执行 B 操作。”



2

系统形成执行逻辑

理解并记录这套规则



3

以后触发 A

环境 / 数据再次吻合



4

自动执行 B

无需人再干预



“

你只需要告诉他：遇到这种情况怎么处理，
系统内部的程序就会自我编程、自我管理。

”

给 AI 加上「器官」(多模态感知)

让智能体像人类一样听到、看到、理解世界。



麦克风

收音

你说话，他能听到。



摄像头 (视频)

看图像

你做的动作，他能看懂。

“

NXNOS 给它集成了器官——麦克风收音、视频看图像。整个智能体就会像人类一样：你说话他能听到，你做的动作他能看懂。

”

NXNOS 5.1 框架：围绕浏览器 + 适配器

现在市面上大部分的软件，都是浏览器。所以这套新框架，是围绕浏览器的使用来展开的。

1 浏览器为核心

浏览器里的大部分工作，都是通过 CLI 接口调用的。



2 集成大量先进经验

5.1 版本集成了非常多的先进经验：人干某个事的第一步、第二步、点哪个按钮。



3 通过适配器扩展能力

NXNOS 已经把这些步骤和经验集成好了。如果有些事 AI 干得不出色，就需要去开发适配器。



“

围绕浏览器 + 适配器，让 AI 能完成电脑上的各种工作。

”

什么是「适配器」？

让 AI 学会完成更多具体的任务。

适配器是什么？

适配器是让 AI 学会操作特定软件或完成特定任务的一种数据封装。

开发适配器，本质上就是对人的「完整操作行为」进行数据标注。



标注的不是某一个动作，
而是你的整个操作行为和流程。

什么时候需要适配器？



当某个 AI 在处理某项任务时表现不够出色，就需要开发适配器。



通过适配器，把人的操作经验转化为 AI 可执行的程序。



这样，AI 就能更加准确、高效地完成这类任务。



“

开发适配器，就是把人的经验变成 AI 的能力。

”

只需干一遍：报税案例

你只需要亲自操作一遍，AI 就会全自动地总结出一个适配器并封装好。



你只要干一遍，AI 就会全自动地总结出一个适配器并封装好。



竞争格局：为什么要开发自己的适配器

掌握自己的适配器，才能在未来的竞争中占据主动。

1 OpenAI 在做什么？

ChatGPT 买了大量的苹果生产的所有电脑，雇了很多人去开发适配器、训练模型，下一步竞争会进入白热化。



大量采购
苹果电脑



雇佣大量
开发人员



开发适配器
训练模型

2 封闭 vs 自主

如果不用自己开发的，只能用别人的：
调用的电费成本 1 元，
加 token 费收你 5 元。



电费成本
1 元

加上 token 费
5 元

3 细分领域的窗口期

对方雇成千上万的人，但数据量太大，
1~2 年内依然没法和我们竞争，
我们依然可以在细分领域称霸。

- ✓ 对方投入巨大
- ✓ 数据量庞大
- ✓ 1~2 年内难以追赶
- ✓ 我们仍有机会

细分领域
称霸

“

掌握自己的适配器，才能在未来的竞争中占据主动，
并在细分领域建立优势。

”

一张图总结 NXNOS 的完整逻辑

从真实世界到数字世界，打通感知、思考与行动，让 AI 真正为你工作。



“ 人在电脑和手机上的所有工作，将全部由NXNOS负责完成 ”